

Learning Lifted Symbolic Planning Domains from Image Sequences

VISA: Visual-to-Symbolic Action Learning

Ulzhalgas Rakhman Paul Schulte Hector Geffner

Motivation: Enabling Generalizable Visual Planning

Symbolic planners require action models

- But these are usually **handcrafted**.
- **Not naturally grounded** in raw *visual observations*.

Prior visual model-learning approaches assume

- A **fixed number of objects**, *limiting generalization*.
- Training traces contain **fully grounded STRIPS actions** with *all arguments specified*.

Objective: Learn lifted domains directly from images, enabling generalization to *larger* planning instances.

Why Is This Difficult?

1. Grounding

- Images contain pixels, not symbolic predicates.
- Object identities and relations are not given.

2. Lifting

- Object counts change across planning instances.
- Fixed propositional encodings do not transfer.

3. Partial Actions

- Traces may omit action arguments.
- They are not full STRIPS action traces.

The learned representation must be *grounded*, *relational*, and *lifted*.

Previous Work: Three Pieces of the Puzzle

LatPlan

- ✓ Learns latent, propositional action models from state image pairs
- ✓ Does not rely on a known symbolic vocabulary
- ✗ Learned model is uninterpretable
- ✗ Latent propositional representation is tied to a fixed object set

ROSAME

- ✓ Learns lifted, human-readable action models from visual traces
- ✓ Computes probabilistic preconditions and effects for action schemas
- ✗ Executed actions are fully observed
- ✗ Requires full STRIPS action traces

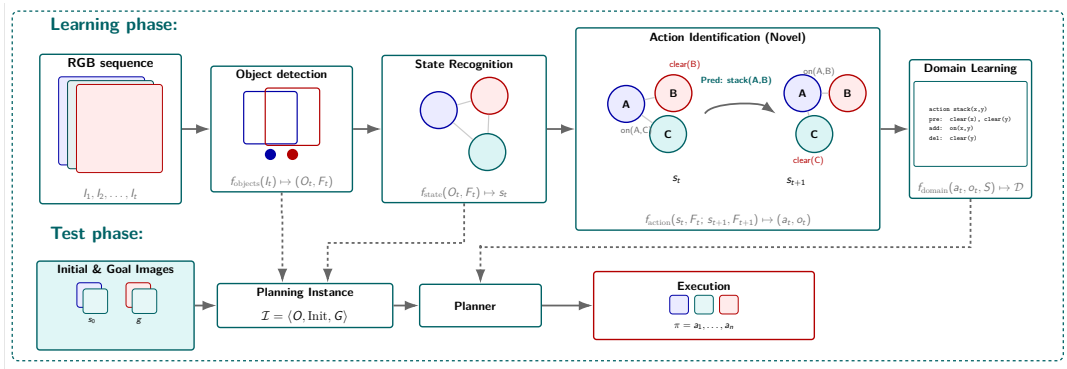
ROSAME v.2

- ✓ Jointly learns to predict states, executed actions, and a lifted action model
- ✓ Removes the need for observed action labels
- ✗ Relies on costly MILP-based correction during training
- ✗ Hard to scale to longer traces

VISA differs by grounding images into *dynamic relational scene graphs* and learning lifted domains from *partially* specified visual *action traces*.

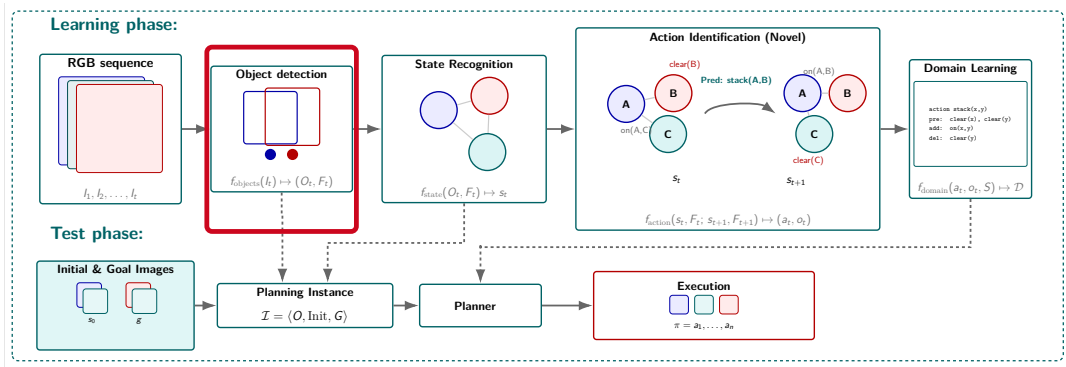
Asai et al., JAIR'22 Xi et al., ICAPS'24 Xi et al., ICAPS'26

VISA at a Glance



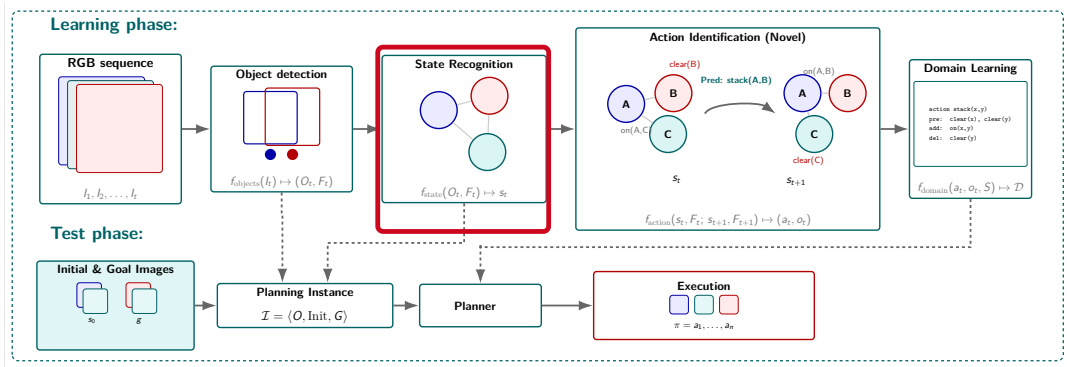
One architecture, two phases: learn a lifted domain from visual traces, then reuse it to plan from new images.

VISA at a Glance

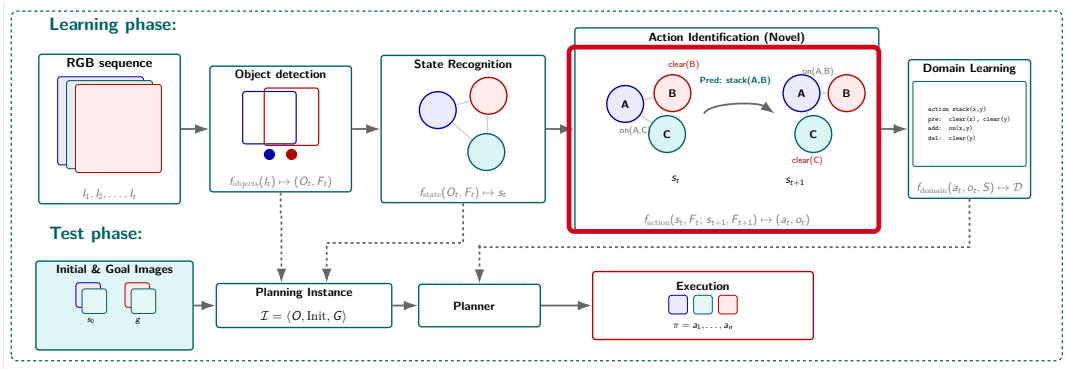


1. **Object detection** turns every frame into a variable-size set of object instances and visual features.

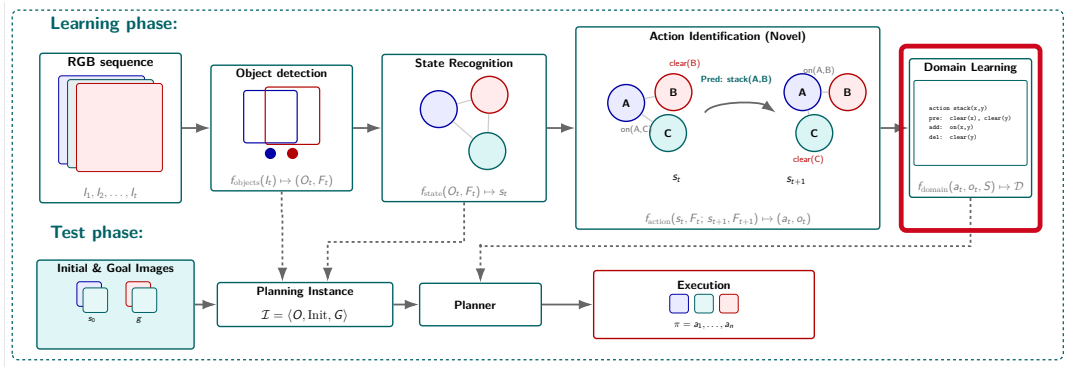
VISA at a Glance



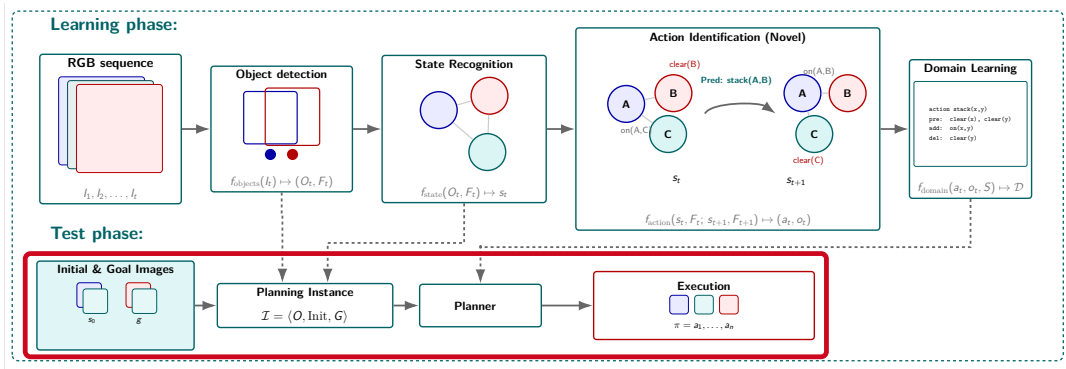
2. State recognition converts detected objects into a relational scene graph interpreted as a symbolic state.



3. Action identification explains each graph transition with an action type and its object arguments.

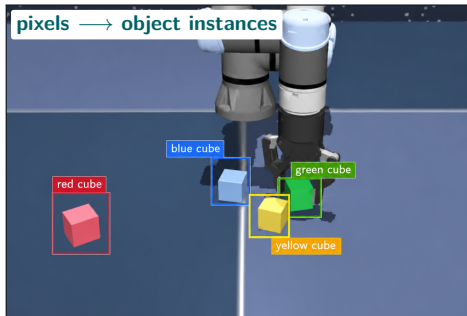


4. Domain learning lifts state-action traces into reusable STRIPS+ action schemas.



5. Planning combines a learned domain with new initial and goal images to produce a plan.

1. Object Detection: Pixels Become Objects

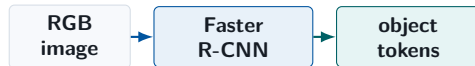


$$I_t \mapsto O_t = \{(\hat{c}_j, \mathbf{b}_j, \mathbf{f}_j)\}_{j=1}^{|O_t|}$$

class • box • feature vector

We use Faster R-CNN

fine-tuned for each domain



- ▶ CNN backbone extracts image features.
- ▶ RPN proposes candidate objects.
- ▶ Detection head predicts class and box.
- ▶ RoI feature is retained as $\mathbf{f}_j$.

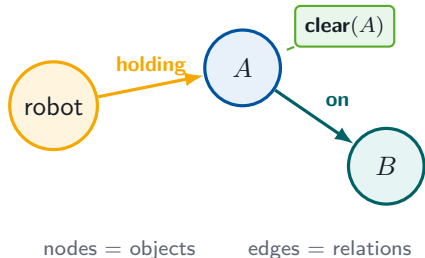
Keep $\mathbf{f}_j$: each object's learned feature is passed to **relation prediction** and **action prediction**.

2. State Recognition: Objects Become a Symbolic State



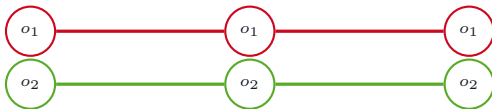
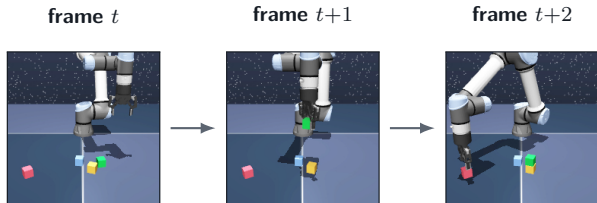
What DSG-DETR does

- Builds features for each **ordered object pair**.
- A spatial transformer reasons over relations within the frame.
- A classifier predicts **unary and binary predicates**.



A scene graph is interpreted directly as the symbolic planning state.

2. State Recognition: Why the Graph Is Dynamic



object tracklets preserve identity

Temporal consistency

- Associate detections across frames.
- Temporal attention refines identities and relations.

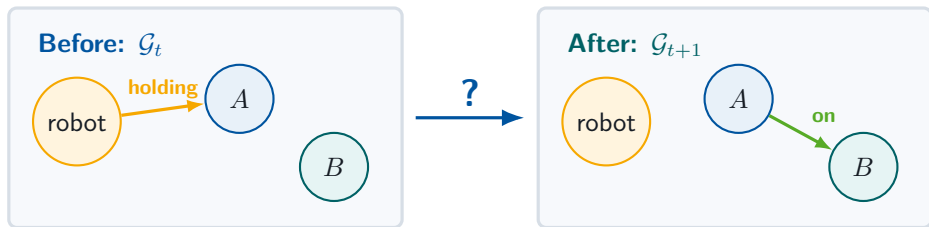
$$\mathcal{G}^{\text{dyn}} = \{\mathcal{G}_1, \dots, \mathcal{G}_T\}$$

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$$

$\mathcal{V}_t$: objects $\mathcal{E}_t$: relations

Planning operates on relational scene graphs, not pixels.

3. Action Identification: What Caused This Transition?



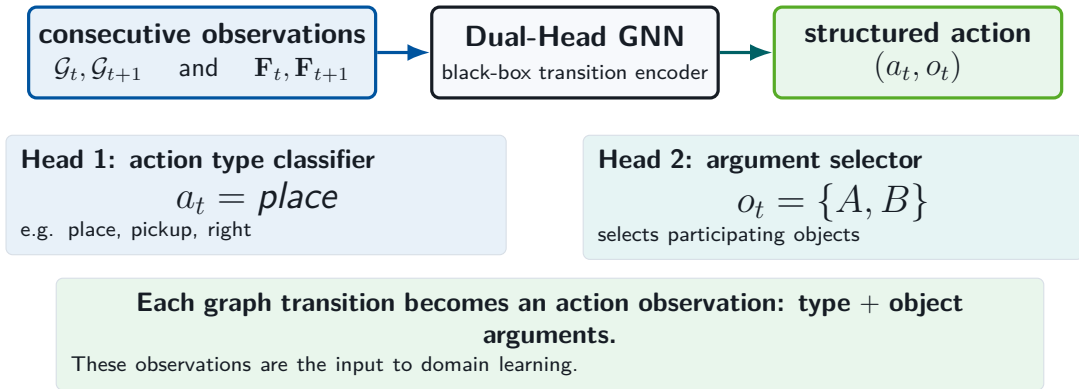
– **holding**(robot, A)
+ **on**(A, B)



structured action
place(A, B)

Action identification is transition explanation: infer the action that best accounts for the changed relations.

3. Action Identification: Dual-Head Structured Prediction

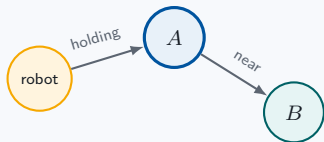


Yan et al., AAAI'18 (ST-GCN)

3. Action Identification: How the Network Reasons

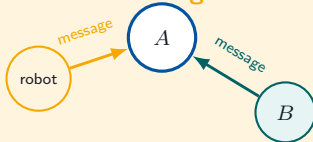
A graph neural network lets each object learn from its relational neighborhood.

1. Initialize nodes



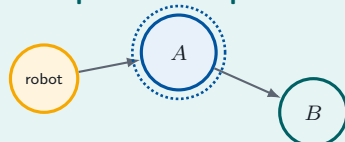
identity + visual features
form initial embeddings $h_v^{(0)}$

2. Pass messages



aggregate incoming messages
from neighbors and relations

3. Update and repeat

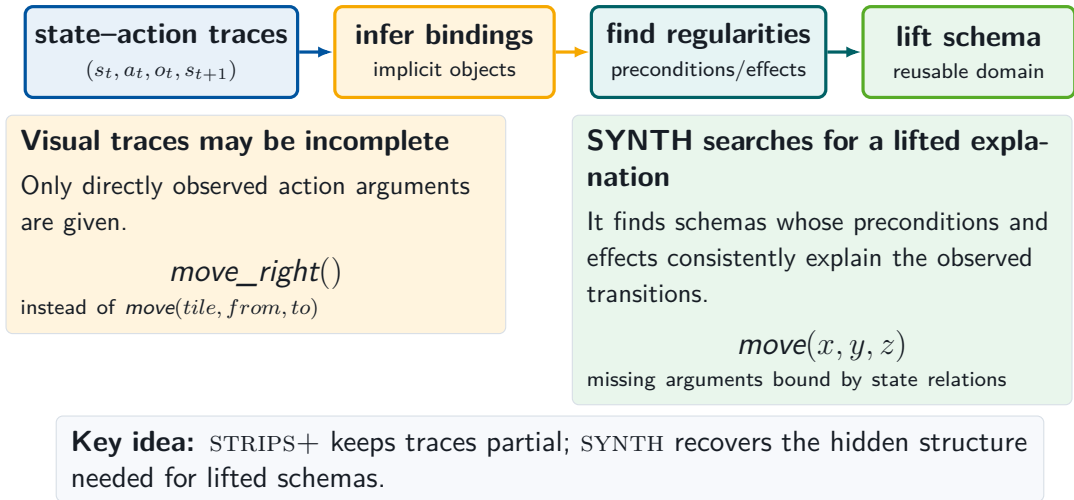


combine feature + messages
into $h_v^{(k+1)}$; repeat K times

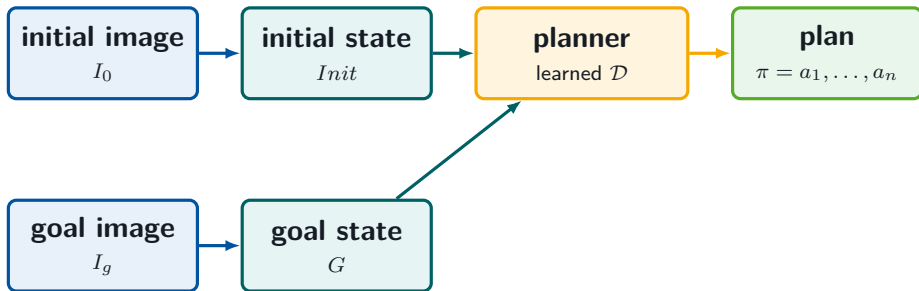
After K rounds, each node represents itself *in context* – not in isolation.

Gilmer et al., ICML'17 (message-passing neural networks)

4. Domain Learning: How SYNTH Recovers Schemas



Planning from Images with the Learned Domain



Training

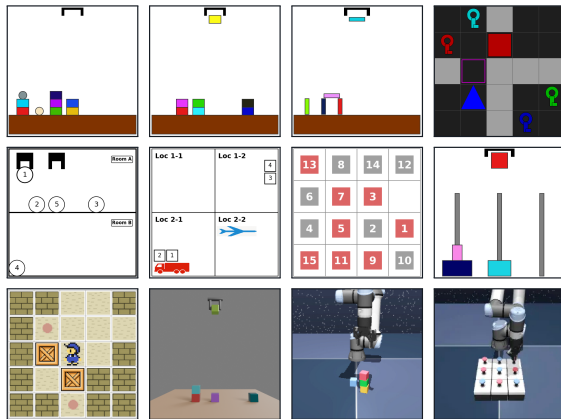
Learn domain $\mathcal{D}$ once from visual demonstrations translated into state-action traces.

Deployment

Translate new initial/goal images into states and solve with a standard symbolic planner.

Automatic dataset generation

- ▶ Use **PDDL**Gym planning domains.
- ▶ Randomly sample an initial and goal state.
- ▶ Solve the planning problem with a classical planner.
- ▶ Execute the plan to generate image sequences.
- ▶ Automatically record all symbolic annotations.



PDDL Gym planning domains (left) and OGBench robotic manipulation environments (right).

Silver & Chitnis, PRL@ICAPS'20 (PDDL Gym) Park et al., ICLR'25 (OGBench)

Main Results

Type	Domain	N_O	N_R	N_A	Objects	Frames	Seq.	mAP	F1	ac_{type}	ac_{args}	VAL
Synthetic	BallHiding	4	8	6	5–12	8.6k	4–19	1.00	.99	100	99.82	1.00
Synthetic	Blocksworld	3	6	4	5–12	8.3k	2–23	1.00	1.00	100	99.35	1.00
Synthetic	BW-Realistic	3	6	4	5–12	8.3k	2–23	.99	.99	100	93.94	1.00
Synthetic	Dominoes	3	13	13	5–7	8.3k	2–16	1.00	1.00	100	100	1.00
Synthetic	Grid	1	16	20	25	5.2k	2–10	1.00	.98	99.65	99.84	1.00
Synthetic	Gripper	3	6	4	9–10	8.1k	2–10	1.00	1.00	100	100	1.00
Synthetic	Logistics	4	3	6	10–11	8.4k	2–17	1.00	1.00	99.73	91.18	1.00
Synthetic	SlidingTiles	3	5	4	18–32	8.8k	2–10	1.00	1.00	100	N/A	1.00
Synthetic	Sokoban-undir.	1	4	2	25	6.8k	4–12	1.00	1.00	100	100	1.00
Synthetic	Hanoi	3	6	2	6–9	10.7k	2–31	1.00	.99	100	100	1.00
Interpol.	Blocksworld	3	6	7	5–12	26.1k	2–31	1.00	1.00	99.95	99.91	N/A
Interpol.	BW-Realistic	3	6	7	5–12	26.1k	2–31	1.00	.99	99.24	96.42	N/A
OGBench	Permuting Cubes	3	4	2	2–3	2.3k	2–6	.88	.73	83.80	70.52	.67
OGBench	Stacking Cubes	3	6	4	2–3	3.5k	2–8	.68	.52	58.18	43.54	OR
OGBench	Puzzle	2	4	2	9	6.2k	2–8	.53	.37	21.30	13.72	OR

OR = object recognition failure.

Generalization (OOD) Results

Domain	O_{train}	O_{test}	$Grid_{train}$	$Grid_{test}$	F1	aC_{type}	aC_{args}	VAL
BallHiding	5–8	10–11	N/A	N/A	.89	99.76	96.25	.80
Blocksworld	4–8	9–11	N/A	N/A	1.00	99.57	100	1.00
BW-Realistic	4–8	9–11	N/A	N/A	.99	96.22	85.90	1.00
Grid	25	36	5×5	6×6	.94	89.51	92.68	.85
Gripper	7–8	9–10	N/A	N/A	1.00	100	100	1.00
Logistics	8–10	11–12	N/A	N/A	1.00	99.23	95.51	1.00
SlidingTiles	18	32	3×3	4×4	.84	97.41	N/A	.80
Sokoban-undir.	25	36	5×5	6×6	.83	79.45	38.08	SR
Sokoban-undir. goals	25/2	25/3	5×5	5×5	.99	99.90	99.90	1.00
Hanoi	6–8	9	N/A	N/A	1.00	100	100	1.00
Blocksworld interpol.	4–8	9–11	N/A	N/A	.98	99.88	97.27	N/A
BW-Realistic interpol.	4–8	9–11	N/A	N/A	.99	98.77	95.64	N/A

SR = state representation not compatible with the target planning formalism.

- ▶ **Perception bottleneck:** Low-resolution or cluttered images cause object-detection errors, which downstream symbolic modules cannot fully recover from.
- ▶ **Supervision bottleneck:** VISA requires supervised annotations for learning visual relations and action identification, although domain learning can use partially observed states and does not require exhaustive STRIPS labeling.
- ▶ **Scalability bottleneck:** State and action modules use attention over object pairs: n objects yield $O(n^2)$ pair embeddings, and attention over these pairs can require up to $O(n^4)$ memory for trajectories.
- ▶ **STRIPS-alignment:** Some learned predicates induce transitions that cannot be expressed with fixed add/delete action schemas.

1. Ground images as object-relation graphs.

2. Predict actions as structured graph transitions.

3. Learn lifted schemas and keep classical planners in the loop.

VISA is a simple, direct approach for learning action models from variable-object image traces.

Thank you.

$$f_{\text{objects}} : I_t \mapsto (O_t, \mathbf{F}_t)$$

$$f_{\text{state}} : (O_t, \mathbf{F}_t) \mapsto s_t = \mathcal{G}_t$$

$$f_{\text{action}} : (s_t, \mathbf{F}_t, s_{t+1}, \mathbf{F}_{t+1}) \mapsto (a_t, o_t)$$

$$f_{\text{domain}} : (\{(a_t, o_t)\}, \{s_t\}) \mapsto \mathcal{D}$$

At test time, initial and goal images are translated into $\langle O, \text{Init}, G \rangle$, paired with the learned domain $\mathcal{D}$, and solved by a symbolic planner.

One-sentence answer

VISA learns a lifted planning domain from visual traces by converting images into relational scene graphs, predicting structured actions between graph states, and learning STRIPS+ schemas from the resulting traces.